

Facundo Goiriz

Lead GenAI Engineer / AI Workflow Engineer

Python automation, LLM agents, API integrations, DSPy, FastAPI, WhatsApp workflows

English C1 | Spanish Native | Portuguese B2

facundogoiriz@gmail.com | [LinkedIn: facundogoiriz](#) | [GitHub: fakamoto](#)

PROFESSIONAL SUMMARY

Applied AI engineer and founder-led builder focused on turning ambiguous automation ideas into working systems: Python scripts and backends, API integrations, LLM workflows, agent tools, WhatsApp/CRM automation, structured outputs, and production deployment.

I currently run Konecta Labs, my own applied AI consultancy, where I work directly with clients from discovery through architecture, prototype, implementation, deployment, and iteration. The work is practical by design: connect the systems the client already uses, build the smallest reliable workflow that solves the business problem, add observability and guardrails, then improve the workflow with real operator feedback.

My strongest fit is AI workflow engineering rather than academic ML research. I have built systems that read messy inbound requests, classify intent, call APIs, retrieve records, generate structured recommendations, write operator-ready responses, schedule follow-ups, transcribe audio, audit conversations, and keep the full trail persisted for review. I am comfortable owning the whole path from prompt/program design to FastAPI endpoints, SQLAlchemy persistence, React/Vite backoffice screens, Docker deployment, runtime debugging, and client communication.

AI WORKFLOW ENGINEER ROLE MATCH

- ▶ Turn loosely defined client needs into working technical solutions: map the process, identify systems and APIs, design the workflow contract, build the first usable version, and iterate from real operator feedback.
- ▶ Build AI-powered workflows for operations, sales, CRM, QA-style conversation auditing, marketing follow-up, education/content generation, quote preparation, intake, and internal process improvement.
- ▶ Connect tools, platforms, and AI services through REST APIs, webhooks, async workers, provider status callbacks, background jobs, and protected internal API routes.
- ▶ Write lightweight Python scripts and production Python services for automation, integration, data extraction, transcription, media conversion, API search, scheduled tasks, and repeatable local workflows.
- ▶ Design and implement AI agents that complete defined tasks with guardrails: classify context, choose an allowed action, call tools, persist state, ask for missing information, hand off to humans, and keep an audit trail.
- ▶ Prototype quickly in ambiguous environments, then harden the parts that prove useful: typed outputs, validation, retries, rate limits, logs, tests, delivery tracking, and operator-facing debugging surfaces.
- ▶ Debug workflow failures across APIs, scripts, AI outputs, webhook delivery, auth boundaries, provider errors, malformed media, database state, and live server/runtime logs.
- ▶ Review and evaluate AI-generated output/code pragmatically: compare it against the contract, run it through tests or replay cases, inspect failure modes, and simplify it into maintainable Python.
- ▶ Comfortable collaborating directly with senior technical leaders and founders while taking ownership of implementation in consulting, startup, freelance, and fast-moving product environments.

CORE AI WORKFLOW ENGINEERING STRENGTHS

- ▶ Python-first workflow automation with FastAPI, async Python, SQLAlchemy, Pydantic, httpx, Typer, PyTest, lightweight scripts, and Docker.
- ▶ LLM application development with OpenAI API, LiteLLM, DSPy, structured outputs, tool calling, embeddings, RAG, classifiers, evaluators, prompt/program iteration, and working familiarity with Claude/Gemini-style APIs.
- ▶ Production automation across WhatsApp, email, CRM backoffices, calendars, public pages, CMS/content workflows, file/audio/media pipelines, and third-party APIs.
- ▶ Agentic systems with durable state, tool manifests, audited side effects, scheduled tasks, memory, human handoff, and operator-visible traces.

- ▶ Multimodal workflows using Whisper/OpenAI transcription, GPT Vision style image understanding, Text-to-Speech, voice cloning, MuseTalk, FFmpeg, S3, and GPU/HPC execution.
- ▶ Practical deployment and operations with Docker Compose, Traefik, health checks, protected internal APIs, runtime logs, live server verification, and failure recovery.
- ▶ Product-minded execution: define the workflow contract, ship a working version quickly, instrument the bottlenecks, collect feedback, and improve the system where it actually fails.
- ▶ Startup adaptability: able to move across client use cases, tools, constraints, and incomplete requirements without waiting for a perfectly structured spec.

TECHNICAL STACK

- ▶ Languages: Python, TypeScript, JavaScript, SQL, Markdown.
- ▶ Backend: FastAPI, SQLAlchemy, Pydantic, SQLite, PostgreSQL, Django, Django REST Framework, Typer.
- ▶ AI / LLM: OpenAI API, LiteLLM, DSPy, function calling, structured outputs, embeddings, RAG, tool calling, eval loops, LLM-as-judge, Whisper, GPT Vision style workflows, Text-to-Speech, Claude/Gemini API exposure.
- ▶ Automation and integrations: REST APIs, webhooks, WhatsApp Cloud API, Twilio WhatsApp webhooks, AgentMail, Calendly-style scheduling, Airtable, S3, custom APIs, CSV/PDF/text intake.
- ▶ Frontend and operator tooling: React, Vite, TypeScript, Next.js, TailwindCSS, Clerk RBAC, backoffice dashboards, internal admin tools.
- ▶ Infrastructure: Docker, Docker Compose, Traefik, uv, Linux servers, SSH, SLURM, BSC MareNostrum 5 / EuroHPC workflows, FFmpeg.
- ▶ Quality and operations: PyTest, QA automation, async worker loops, retries, rate limiting, delivery tracking, persistent audit trails, structured logs, health checks, live runtime verification.

SELECTED AI WORKFLOW CASE STUDIES

Production CRM, WhatsApp Automation, and AI Backoffice - Konecta Labs / Contadores

Built and operated a production FastAPI backoffice for CRM, WhatsApp automation, Workstation client delivery, and operator tools. The system combines lead intake, conversation state, autonomous AI replies, WhatsApp delivery tracking, internal APIs, public trial pages, media handling, and a React/Vite operator interface.

- ▶ Built the backend around FastAPI, SQLAlchemy, SQLite, Pydantic, protected internal routes, static frontend serving, runtime readiness endpoints, and Docker Compose services for backend, bot, and Traefik.
- ▶ Designed persistence for leads, messages, alerts, agent runs, scheduled tasks, workstation clients, public pages, media assets, CRM threads, emails, contacts, and tasks, with SQLite WAL/busy-timeout tuning for concurrent local/server operation.
- ▶ Implemented a WhatsApp and AgentMail bot runtime that persists inbound webhooks, replays delivery from a durable local inbox, tracks outbound provider status, reconciles booking events, and runs a continuous worker loop.
- ▶ Built DSPy-driven conversation programs with a fixed action contract, structured output normalization, classification, reply generation, post-video reply handling, and guardrails for when to answer, ask follow-up questions, hand off, pause, or close a lead.
- ▶ Automated marketing and sales follow-up workflows where the AI must keep selling, answer objections, route approvals, avoid duplicate replies, and preserve the full CRM state for manual operators.
- ▶ Added an autonomous Codex agent runtime with durable memory, scheduled tasks, tool manifests, audited side effects, run context snapshots, and product-facing tools for WhatsApp replies, domain lookups, lead movement, Workstation actions, and memory reads/writes.
- ▶ Shipped a Workstation solo-page flow for paid clients: CRM-to-Workstation conversion, preview generation, public trial URLs, image/media handling, approval/handoff states, scheduled heartbeats, and operator-visible progress.
- ▶ Implemented OpenAI-based audio transcription for WhatsApp media, including unsupported-format handling, OGG/Opus conversion through FFmpeg, and default use of gpt-4o-transcribe.
- ▶ Added production safeguards: auth boundaries between operator and internal bot routes, no-store runtime surfaces where needed, delivery failure acknowledgement, requeue paths, and live deploy verification through server logs and protected endpoints.
- ▶ Built tests around the main backend, Workstation flows, conversation bot, and worker loop, including more than 150 backend tests in the main CRM test file plus additional bot/workstation suites.

DCLATAM Quote Bot - Konecta Labs

Built a production quote assistant for DCLATAM, a LATAM/US technology solutions company, to help sales operators convert raw customer requests into stock-aware commercial recommendations.

- ▶ Implemented a FastAPI service titled DCLATAM Cotizacion Bot API with authenticated access, a web operator interface, background jobs, SQLAlchemy persistence, and a Docker deployment path.
- ▶ Built the core DCLatamQuoteBot as an async DSPy program that receives a customer request, generates search queries, calls DCLATAM stock APIs, analyzes product availability, and returns a quote plan, markdown table, reasoning, and raw stock results.
- ▶ Integrated against the DCLATAM dashboard API with environment-based credentials, token handling, product search parameters, stock filters, pricing data, and response normalization.
- ▶ Designed the workflow for real sales use: paste a customer request, enqueue a quote job, show processing state, consult stock, present alternatives, and produce an operator-ready response.
- ▶ Added traceability by persisting the generated query, stock reasoning, markdown table, quote plan, raw stock JSON, and operator feedback.
- ▶ Ran optimization experiments with DSPy GEPA, LLM-as-judge scoring, baseline evaluation, feedback datasets, and sync wrappers around the async production program.
- ▶ Addressed practical workflow failures such as overly narrow product queries, missing alternatives when exact stock was unavailable, poor stock prioritization, and responses that were not clear enough for sales operators.
- ▶ Treated the bot as a practical operations automation system: it connected external product data, AI reasoning, background processing, operator review, and feedback loops into one workflow.

Avatar4University / eCampus AI Course Platform - Konecta Labs

Built an AI-assisted course and avatar platform for an Italian education initiative, with organization-level course management, role-based access control, lesson generation, slide generation, audio/video pipelines, and deployment infrastructure.

- ▶ Built a FastAPI + DSPy backend and Next.js frontend for creating and managing organizations, courses, lessons, slide templates, and user permissions.
- ▶ Implemented role-based access control with owner, admin, manager, member, and viewer permissions, including organization-level course access rules.
- ▶ Generated lessons automatically with the OpenAI API and converted course material into markdown slides, PPTX files, and audio-supported learning assets.
- ▶ Implemented multimodal pipelines for content, slides, PDFs, audio, and avatar video using OpenAI/LiteLLM, Whisper, MuseTalk, FFmpeg, S3, and GPU/HPC workflows.
- ▶ Integrated BSC MareNostrum 5 / EuroHPC execution patterns with SLURM jobs, SSH tunnels, and runtime playbooks for long-running avatar/video workloads.
- ▶ Built production and development Docker paths with separate local hot reload and optimized production modes.
- ▶ Worked across product architecture, implementation, client communication, and operational debugging for a high-complexity education platform rather than a narrow AI demo.

Konecra Auditor - Conversation Audit and Report Automation

Built a stage-based conversation-auditing backend for evaluating how companies respond across chat, WhatsApp, and email channels, then producing structured reports and PDF-ready artifacts.

- ▶ Implemented the product around a clear FastAPI + SQLAlchemy + DSPy pipeline: URL to contact discovery, conversation/objective to next reply, company context to structured JSON report, structured report to PDF content model, and PDF model to on-demand PDF binary.
- ▶ Designed batch intake that accepts pasted text and uploaded files, converts sources into labeled text blocks, extracts company URLs, creates company records, and keeps multiple company scans in flight.
- ▶ Built operator and bot routes for company scanning, contacts, message transcripts, delivery status, report generation, artifact retrieval, CEO audit delivery, reply inbound webhooks, and tracked senders.
- ▶ Persisted structured report snapshots, rendered HTML, report windows, delivery state, contacts, messages, archived contacts, and PDF models in the database.
- ▶ Added primitive backoffice auth with cookie sessions for operators and internal-token protected routes for bot/runtime callers.
- ▶ Kept the implementation contract-first: each stage has explicit inputs and outputs, AI outputs are persisted as structured data, and PDF rendering is separated from report reasoning.

- ▶ Covered a QA automation use case by evaluating response quality, tone, speed, accuracy, commercial skill, follow-up behavior, and final report readiness from real conversations.

WhatsApp-Native Conversational Automation Prototypes

Built and explored several WhatsApp and conversational workflow products that replace rigid web forms with guided chat, tool execution, and operator-facing state.

- ▶ Built Chatterface, a FastAPI WhatsApp bot using Twilio webhooks for conversational data collection and tool-call execution.
- ▶ Worked on real estate and restaurant workflows where WhatsApp conversations collect intent, capture order/intake details, persist state, and trigger follow-up actions.
- ▶ Used pywa / WhatsApp Cloud API patterns, Twilio WhatsApp webhooks, Airtable persistence, SQLite/SQLModel state, and DSPy conversation logic across these prototypes.
- ▶ Focused on workflows where the AI must gather missing information, call the right tool, avoid over-automating ambiguous cases, and preserve a transcript for human review.

AI Developer Tools and Prototypes

Built smaller tools and experiments that sharpened the same workflow-engineering skills used in production systems.

- ▶ Built Agentics, a lightweight OpenAI-oriented library for structured outputs, function tools, embeddings, and simpler LLM task composition.
- ▶ Built vision and catalog prototypes with FastAPI, OpenAI, and instructor-style structured extraction.
- ▶ Built CLI and developer tools with Typer and Python for workflow automation, data extraction, and repeatable local execution.
- ▶ Compared AI agent SDKs and local agent execution patterns, including file-backed thread/conversation reuse for continuation across runs.
- ▶ Used AI-generated code as an implementation accelerator while still validating behavior with local runs, tests, simplified diffs, and manual inspection before folding changes into production workflows.
- ▶ Used these projects to test simple abstractions before folding the useful patterns into client systems.

EXPERIENCE

Co-Founder & CTO - Konecta Labs (Own Consultancy)

06/2024 - Present

- ▶ Run a founder-led applied AI consultancy delivering custom GenAI products for international clients, with responsibility for architecture, backend implementation, LLM orchestration, workflow design, deployment, and stakeholder communication.
- ▶ Turn client ideas into working systems by mapping the real business process, defining the automation contract, identifying required APIs/data sources, building the first useful workflow, and iterating from operator feedback.
- ▶ Build Python services with FastAPI, SQLModel, SQLite/PostgreSQL, Pydantic, async workers, OpenAI/LiteLLM, DSPy programs, Docker Compose, Traefik, and React/Vite or Next.js frontends where operator tooling is needed.
- ▶ Write lightweight Python scripts for scraping/extraction, API calls, media conversion, evaluation runs, deployment checks, data cleanup, and one-off automations when a full product surface is unnecessary.
- ▶ Design AI programs that classify messages, extract structured fields, call tools, retrieve records, generate responses, audit conversations, build reports, transcribe audio, and decide when to ask for clarification or hand off to a human.
- ▶ Implement durable automation instead of isolated prompts: database state, idempotency, delivery tracking, retry loops, audit logs, internal tokens, health checks, feedback capture, and production verification.
- ▶ Lead technical product discovery directly with clients, explain tradeoffs in non-academic language, and translate ambiguous requests into implementation plans that a small team can ship.
- ▶ Maintain a house engineering method optimized for readable code: uv, FastAPI, SQLModel, DSPy, Pydantic, explicit stage contracts, simple folder layouts, Docker Compose, and E2E validation.
- ▶ Shipped or developed systems across education, sales quoting, CRM, WhatsApp automation, conversation auditing, real estate training, legal workflow support, restaurants/order intake, and AI sales operations.
- ▶ Built sales and discovery material for AI services by turning transcripts, client notes, product demos, and proof points into usable proposals, Loom scripts, ICP messaging, and case-study assets.
- ▶ Move across unrelated client use cases quickly while preserving engineering quality: clarify the workflow, choose the smallest useful implementation, ship, debug, and harden only what proves valuable.

AI Engineer & Team Lead - Network Ninja Inc

09/2023 - 05/2024

- ▶ Founded the AI function inside the company, shipped multiple AI products end-to-end, and grew the team while partnering directly with the CEO on roadmap, prioritization, and execution.
- ▶ Built production LLM systems with OpenAI, function calling, structured outputs, embeddings, vector databases, RAG, and agent-style workflows for chat, retrieval, and workflow automation.
- ▶ Designed multi-step AI workflows that combined user input, business context, tool calls, memory/state, retrieval, and human review, using patterns similar to LangChain and LangGraph architectures.
- ▶ Built FastAPI services that handled high-volume AI API usage, including async Python, concurrency controls, retries, rate limiting, background work, and latency/failure monitoring.
- ▶ Worked in fast-moving product contexts where requirements changed often, so prototypes needed to be small, testable, easy to debug, and easy to reshape after stakeholder feedback.
- ▶ Added observability and feedback loops for prompt iteration, quality review, latency tracking, failure analysis, and product-specific improvements, using tools and patterns such as Langfuse-style tracing.
- ▶ Shipped voice and multimodal features with Whisper, GPT Vision style workflows, Text-to-Speech, voice cloning, MuseTalk, and video/audio processing.
- ▶ Built and improved computer vision workflows, including custom YOLO-style recognition systems and image-recognition work tied to field marketing and retail execution.
- ▶ Worked on applied AI/ML tools used in operational field marketing workflows and image recognition work connected to Kraft Heinz retail execution use cases.
- ▶ Led technical planning, reviewed implementation quality, mentored developers, and translated experimental AI ideas into usable product features.

Mid Python Developer - Dyvenia

09/2022 - 12/2023

- ▶ Built backend and data systems for automation and analytics, and contributed as part of Dyvenia's initial AI integration team.
- ▶ Developed FastAPI REST APIs, data models, and processing services using SQLAlchemy, SQLModel, Docker, and Python.
- ▶ Built data pipelines that transformed raw user and product data into reliable datasets for downstream automation and analytics.
- ▶ Built two open-source tools to support the team's early AI integrations: CLAIX and an OpenAI API wrapper.

Junior Python Developer - Cattler

07/2020 - 03/2022

- ▶ Worked with Django, Django REST Framework, PostgreSQL, and GCP on backend features and data workflows.
- ▶ Contributed to AI image-recognition and GIS-related functionality, including workflows where visual data and location context had to be converted into useful product behavior.
- ▶ Built early production experience around APIs, databases, cloud deployment, debugging, and team delivery.

HOW I BUILD AI WORKFLOWS

- ▶ Start with the operator workflow, not the model: what arrives, who needs the output, what systems must be touched, what can go wrong, and what needs human approval.
- ▶ Define a typed contract before writing AI logic: inputs, output schema, allowed actions, downstream tools, failure modes, and persistence requirements.
- ▶ Keep semantic work inside the AI program and deterministic cleanup in Python: normalize model outputs, validate fields, enforce allowed states, and avoid hidden prompt-only behavior.
- ▶ Build the first version as a thin vertical slice: one real input path, one database record, one model call, one tool call, one operator-visible result.
- ▶ Add feedback and traceability early: prompts/program version, input text, tool calls, raw model output, normalized result, operator edits, delivery status, and failures.
- ▶ Evaluate generated code or model-created implementation suggestions like any other untrusted change: read it, simplify it, run it, test it, and keep only the part that improves the workflow.
- ▶ Prefer small, readable modules with explicit orchestration over clever abstraction. The goal is a workflow that a client or teammate can reason about during production debugging.
- ▶ Verify with the actual runtime path when possible: local tests are useful, but webhook delivery, auth, bot logs, API tokens, provider responses, and server health are what prove the workflow works.

ADDITIONAL PROJECT AND DOMAIN EXPERIENCE

Education and Training Platforms

- ▶ Built AI course-generation workflows that create lesson structure, slides, audio, and avatar-supported educational content.
- ▶ Worked on onboarding and training products that combine typed steps, quizzes, exercises, checkpoints, and repeatable learning paths.
- ▶ Built role-based organization workflows so different users can create, edit, review, or view course material safely.

Sales, CRM, and Lead Automation

- ▶ Built AI systems for inbound lead response, message classification, WhatsApp follow-up, booked-call handling, manual handoff, and public page delivery.
- ▶ Designed bot behavior that distinguishes intake, concrete revision material, approval, silence, no-action cases, and escalation to a human.
- ▶ Built workflows where automation can be paused or stopped per lead without destroying useful CRM state.
- ▶ Built marketing automation support around follow-up timing, personalized responses, public trial links, sales context, and operator-visible lead status.

Conversation Auditing and Quality Reporting

- ▶ Built workflows that act like a customer, contact a company, follow the conversation, score the response, and generate structured reports.
- ▶ Used AI to evaluate tone, speed, accuracy, commercial skill, follow-up quality, and overall customer experience.
- ▶ Converted report reasoning into persisted JSON and PDF-ready content models.
- ▶ Applied QA automation patterns to conversations: define the quality rubric, gather the transcript, evaluate the behavior, persist the score, and generate a report that a business user can act on.

Media, Voice, and Multimodal AI

- ▶ Implemented speech-to-text flows for WhatsApp audio and uploaded media using the OpenAI transcription API.
- ▶ Built audio/video processing paths with FFmpeg, Whisper, voice cloning, MuseTalk, avatar clips, and generated course assets.
- ▶ Worked on image-recognition and product/retail visual inspection workflows with computer vision and structured extraction.

Developer Tooling and Internal Automation

- ▶ Built local scripts and tools to automate research, extraction, content transformation, experiment comparison, thread continuation, and repeatable developer workflows.
- ▶ Used uv-centered Python projects and minimal scripts when the task did not justify a larger framework.
- ▶ Built internal documentation and skills/runbooks so successful patterns could be reused across repos and projects.

SELECTED TECHNOLOGIES BY USE CASE

- ▶ Workflow backends: FastAPI, async Python, SQLAlchemy, SQLite, Pydantic, Docker Compose, Traefik.
- ▶ LLM programs: DSPy, OpenAI, LiteLLM, structured outputs, Chain-of-Thought modules, signatures, evaluators, LLM-as-judge.
- ▶ CRM and messaging: WhatsApp Cloud API, Twilio WhatsApp, AgentMail, webhooks, delivery status tracking, replay queues, operator backoffices.
- ▶ Retrieval and data: embeddings, vector databases, RAG, API search, multi-query search, CSV/PDF/text extraction, structured persistence.
- ▶ Media AI: Whisper, OpenAI transcription, GPT Vision style image workflows, Text-to-Speech, voice cloning, MuseTalk, FFmpeg, S3.
- ▶ Frontend/operator tools: React, Vite, TypeScript, Next.js, TailwindCSS, Clerk RBAC.
- ▶ Operations: Linux, SSH, server deploy scripts, logs, health checks, internal tokens, protected endpoints, SLURM, GPU/HPC execution.

EDUCATION

UAE University, Data Science

03/2018 - 03/2020

- Studied data science foundations including data analysis, data management, NumPy, Pandas, statistics, and neural network concepts with PyTorch.
- Built the academic base later applied in practical Python, backend, AI automation, computer vision, and LLM product work.